POLITÉCNICO DE LEIRIA | ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

Undergraduate in Games and Multimedia | 2025/2026

Computer Graphics

REPORT

Project1- Model Loading

Students

Dinis Stasyuk | 2230673

Introduction

The main goal of this project is loading models with textures, rendering them while allowing to visualise them in a 3D scene with FPS style camera. For this project I used assimp for the model loading, since it was recommended for beginners, included the following external libraries: Sdl3, Glad and OpenGl. And I had to install the DS4Windows to test my controller input.

Implementation

Model Loading and Processing

In the current implementation we have 2 main classes Model responsible for importing the model as whole, then dividing its meshes and its children, providing them to the Mesh class for processing, stores the meshes and draws them after being processed in the Mesh. The Model class also contains logic for matrix transformation for the models.

Mesh class is responsible for preparing the meshes for drawing, so the logic for extraction of textures or their paths from the material and loading them, assigning VAO, VBO, EBO and vertices.

I separated the file import and processing since in the beginning I had reoccurring issues regarding files with more than one mesh, such as only load the first/render the first mesh or only the first texture would load on top of the whole model, also separating them makes code cleaner. Sometime later I watched a **video** from *OGLDEV*, and decided to make the importing and processing fixing the previous issues. Though further down the line it caused issue with the storage of the meshes texture, where the texture would delete themselves before drawing, which my solution was removing the cleaners.

RenderingPipeline

ComputerGraphicsProject1 Main---> Setups window loads Glad, SDL and OpenGl, creates model example modelTest1 than model modelTest1.LoadModel("t1.fbx");

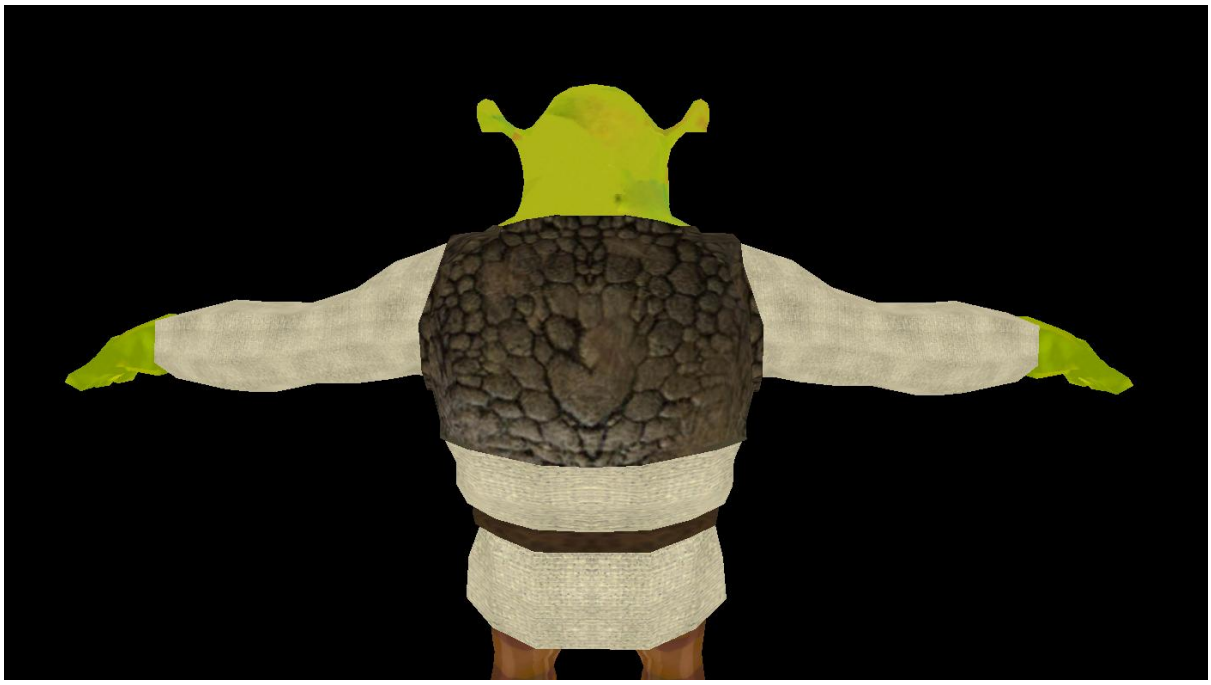
Model LoadModel---> imports model from file than ProcessModel separate the file into multiple Meshes, ProcessMesh and store it; ProcessMesh() extract vertices and textCoords try to get texture from material with loadTextureFromMaterial;

Mesh loadTextureFromMaterial---> usually gets path to texture and then tries to loadTextureFromFile if any fails assigns failure texture;

Mesh ProcessMesh---> once texture is assigned setupMesh where it generates VAO, VBO, EBO and assigns vertices and textCoords.

ComputerGraphicsProject1 Main---> After loading the models creates shaders than loop runs clears window, uses shader, updates matrices and draws models.

Current Rendering Results



ModelLink1

ModelLink2

Shaders

I have the shaders done the same way as in class, though I changed the vertex shader attribution of variables now having it fixed with hardcoded locations 0 position, 1 normPosition, 2 textCoord. So, in the current implementation we have a Shader class which opens the vertex and fragment shader responsible for handling geometry, position and color. Links and compiles those checking for errors, allowing them to convert all geometry, position and color data into pixels which update every frame.

Camera

Camera takes input from controller, keyboard and mouse, their input is checked in the following functions processController, processKeyboard and ProcessInput, worth noting that all the movement speed/sensitivity takes deltaTime into account. I will now explain what these functions do:

-processKeyboard handles movement of the camera it changes the position based on what input was provided. Camera can move in horizontal axis as well as up and down with the SpaceBar and either of the Controll keys respectively. Contains Quit with Escape key, all of the inputs are checked with keyState = SDL_GetKeyboardState. And inside of itself updates the process controller.

-processInput needs SDL_Event provided in the loop for the input. processInput takes controls the FOV with mouse scroll, changes mouse orientation based on motion, clamps pitch rotation for a more FPS like camera.

-processController first triggers the function openController when the bool gotController is false, openController attempts to get the first Gamepad, if it succeeds than controller input can be processed. But before processing the input of the controller we setups deadZones and axisMax needed to ignore minor movement and normalize movement, then we use both joystick 2D axis the right one for movement and the left for camera orientation. Uses triggers to move up and down right one Up and left Down.

Camera also has a getter for FOV and Matrix for the view and projection.

Important when I say camera move or orientation, I mean it transforms all objects not the Camera itself since we are using OpenGl.

Conclusion

Most things are working as intended tough I don't have the mesh cleaner working, normal map and nor the lamp shader.

References

OGLDEV. *Loading models using ASSIMP // OpenGL Tutorial #18* [Video]. YouTube. https://www.youtube.com/watch?v=sP_kiODC25Q

The Graphics Guy. *OpenGL with C++ 7: Loading Models* [Video]. YouTube. https://www.youtube.com/watch?v=CX6cjV2FIgQ

LearnOpenGL – Textures de Vries, J. *Textures*. LearnOpenGL. https://learnopengl.com/Getting-started/Textures

Simple Direct Media Layer. (n.d.). *SDL_GamepadButton*. SDL Wiki. https://wiki.libsdl.org/SDL3/SDL_GamepadButton

As well as the materials provided in class